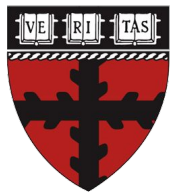
Lecture 21: Automating Software Development: CI/CD with GitHub Actions and other tools II

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Local Git Hooks

- In the previous lecture, we explored local Git hooks. Scripts that run on your own machine.

- They help automate checks before committing or pushing code.

- Examples:
  - **pre-commit:** Runs before code is accepted on the server. Ensures commits meet standards or pass tests.

  - **pre-push:** Runs before pushing commits to the remote repository. Used to check tests, code quality, or sync with remote branches.

Local Git Hooks

- But local hooks have a limitation: they only run on the developer's system.

- This means local automation can be inconsistent across the team.

So, how do we make sure these checks happen for everyone across the entire team or organization?

That's where **remote hooks** come in.

Outline

- **Remote Git Hooks**
- Github Webhooks
- Continuous Integration
- Continuous Deployment (Lec21)

Remote Git hooks

Remote Hooks are executed on the server side (e.g., GitHub Actions, GitLab, or other remote servers). They are enforcers across the entire organization.

Let's look at some examples:

pre-receive: Runs before code is accepted by the repository. Commonly used to enforce standards, run tests, execute linters, or send notifications.

Remote Git hooks

update: Similar to pre-receive, but runs on a per-branch basis. Useful for branch-specific policies or checks.

post-receive: Runs after code has been integrated into the repository. Often used to run integration tests, send notifications, or perform maintenance tasks like purging large files.

Remote Git hooks

- Remote hooks help enforce rules on the server
- But what if we want to trigger actions outside Git — like notifying Slack, updating a dashboard?

That’s where Webhooks come in.

Outline

- Remote Git Hooks
- **Github Webhooks**
- Continuous Integration
- Continuous Deployment (Lec21)

GitHub Webhooks

- Webhooks are similar to hooks in that they trigger actions after specific events (e.g., a push).

- Unlike hooks, webhooks communicate with external services outside of GitHub.

- Some examples include
  - Triggering external CI pipelines (i.e., Jenkins)
  - Deploying code to a production server.
  - Sending notifications (Slack, Discord, etc)

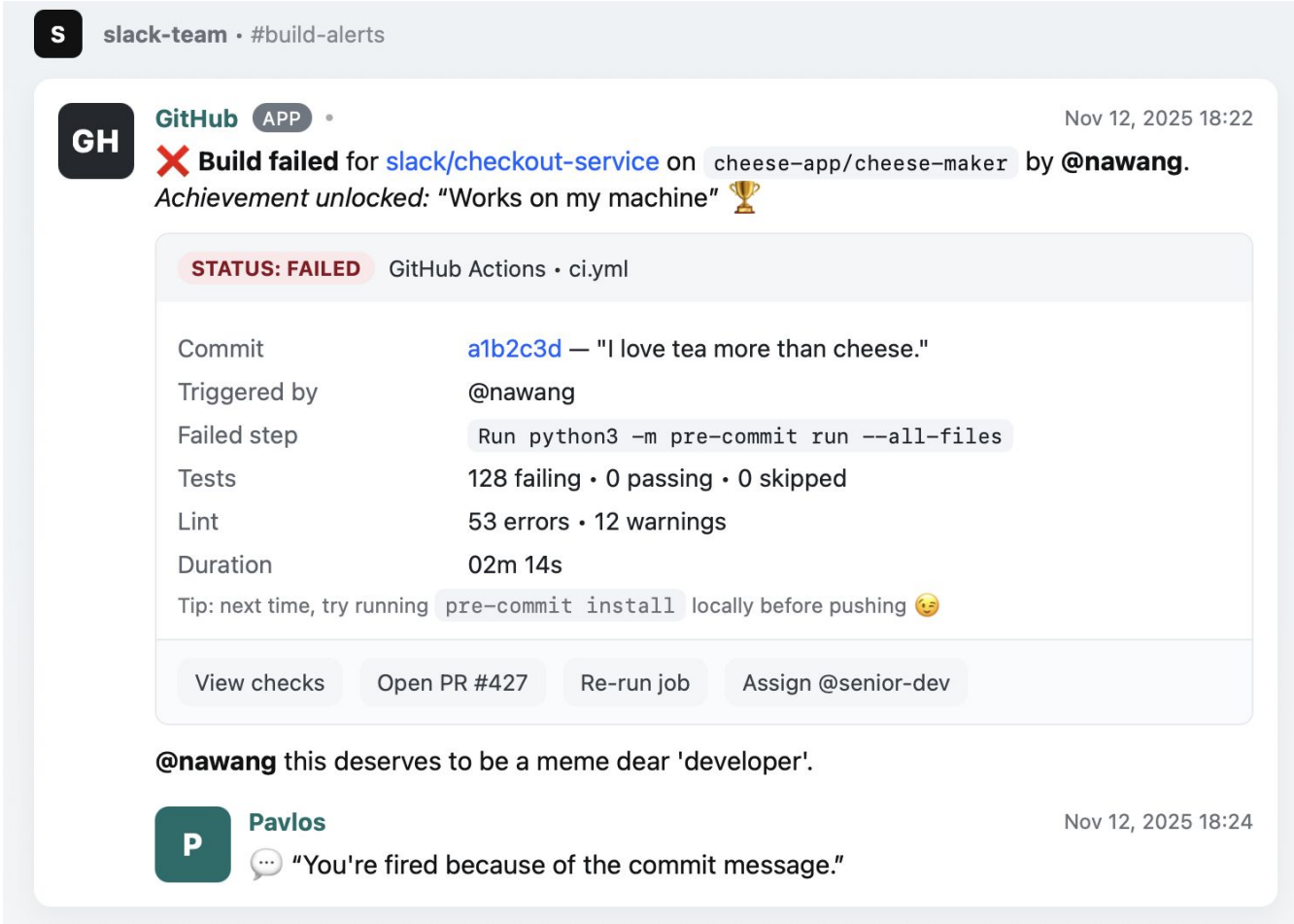
GitHub Webhooks: Example

Now, we don't actually have a Slack webhook set up here…
but let's imagine we did.



GitHub Webhooks: Example

Whenever someone pushes code to GitHub, the webhook automatically sends a message to our team's Slack channel.



GitHub Webhooks: Example

That's the idea, webhooks send real-time updates outside GitHub.

They can alert us, trigger builds, deploy code, or even 'shame' us when something breaks.

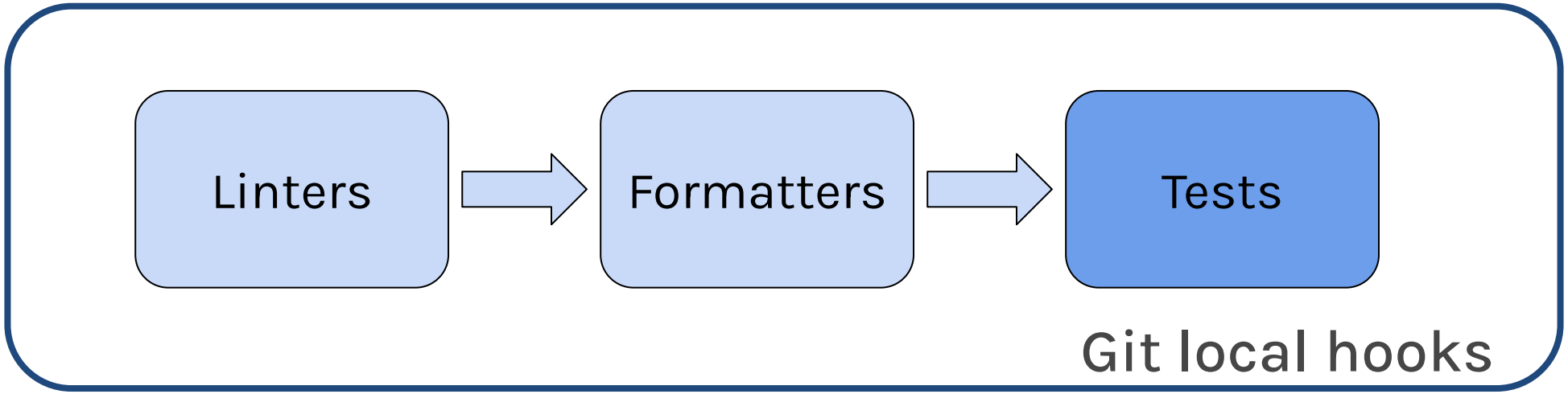


Outline

- Remote Git Hooks
- Github Webhooks
- **Continuous Integration**
- Continuous Deployment (Lec21)

Motivation

We have learned how Git local hooks can help automate checks on our local machine.



All of this happens locally. It's triggered automatically every time the developer tries to make a commit.

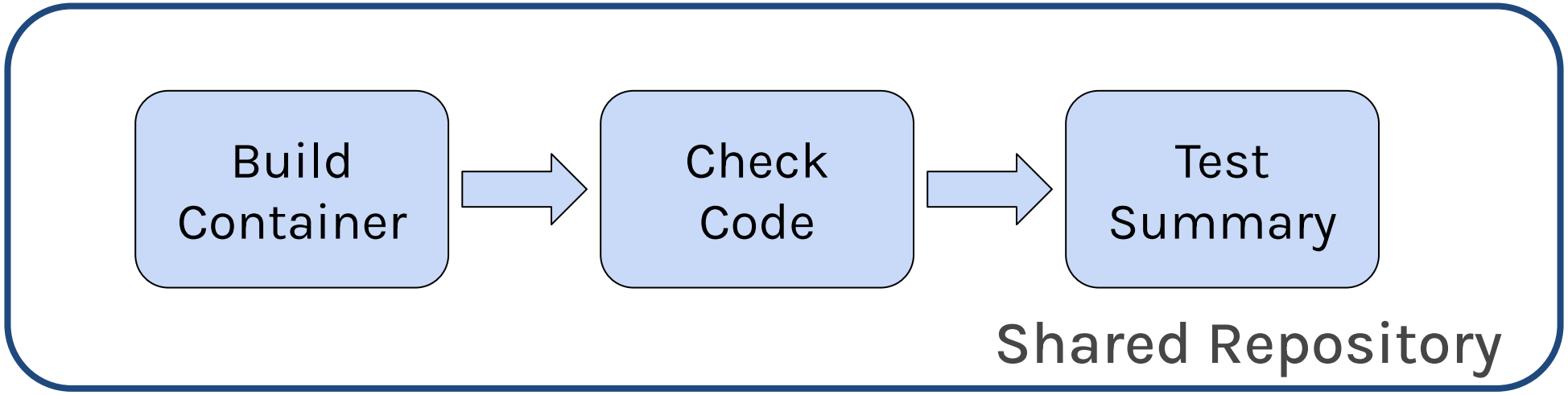
Motivation

But once the code is pushed to the shared repository, the local hooks stop working.

So, we need a centralized and automated way to perform these checks.

Motivation

That's where Continuous Integration (CI) come in. It automates this entire process.



Continuous Integration (CI)

- In traditional software development CI implies
  - faster release cycles,
  - improved code quality, and
  - easier coordination between developers.

- In MLOps, there is an extra layer of complexity.



Continuous Integration (CI)



You might be wondering, what does that mean?

It simply means there's uncertainty at many stages:

- In how models are trained and behave (model development)
- Data that keeps changing.
- Testing, where results may vary.
- And during deployment, where models can drift over time.

Continuous Integration (CI)

How do we address this uncertainty?

1. **Version Everything**

Track versions of code, data, and models so results can be reproduced later.

Tools: **DVC**, **MLflow**, or **Git-based tracking.**

2. **Automate Retraining and Testing**

Set up pipelines that retrain and validate models automatically when data or code changes.

Continuous Integration (CI)

How do we address this uncertainty?

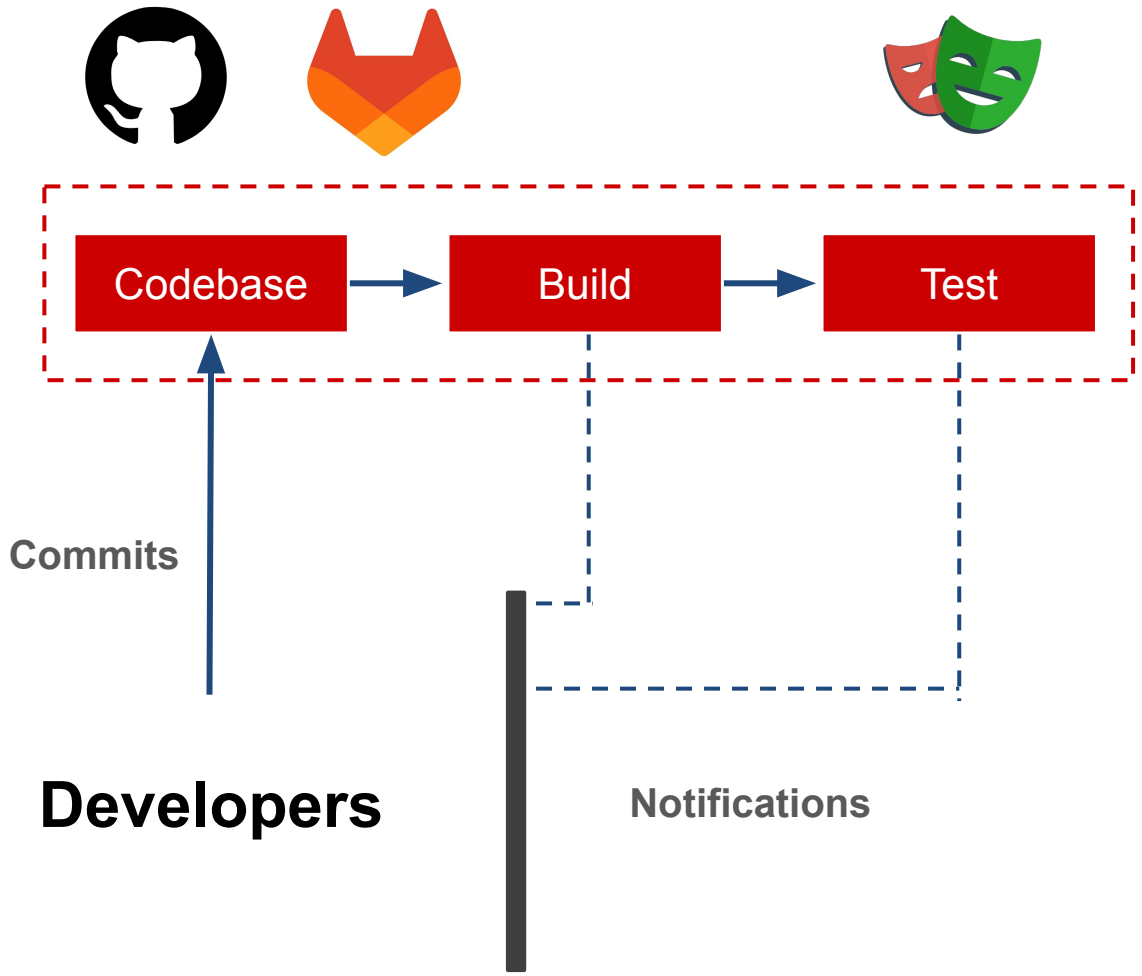
3. **Monitor in Production**

Continuously track model performance, input data quality, and drift after deployment.

4. **Use Controlled Deployment**

Apply safe rollout strategies before full release (we'll look more into this later).

Continuous Integration (CI)



Continuous Integration (CI)

Continuous Integration (CI): Execute a series of scripts automatically, anytime changes are made.

- Continuously Integrate your changes
- Automated testing
- Ensure coding standards
- Static code analysis

Continuous Integration (CI)

- CI in ML means regularly merging code, data, and models into a shared repository.

- Each merge triggers automated tests to ensure new changes don't break the system.

- Fast detection of issues in code, data, or models is crucial delays can cost both time and money.

Continuous Integration (CI)

Challenges:

- Finding the right balance between frequent integration and the high computational cost of model training and testing.

- The setup and frequency of CI often depend on the organization's resources and workflow.

Continuous Integration (CI): Example

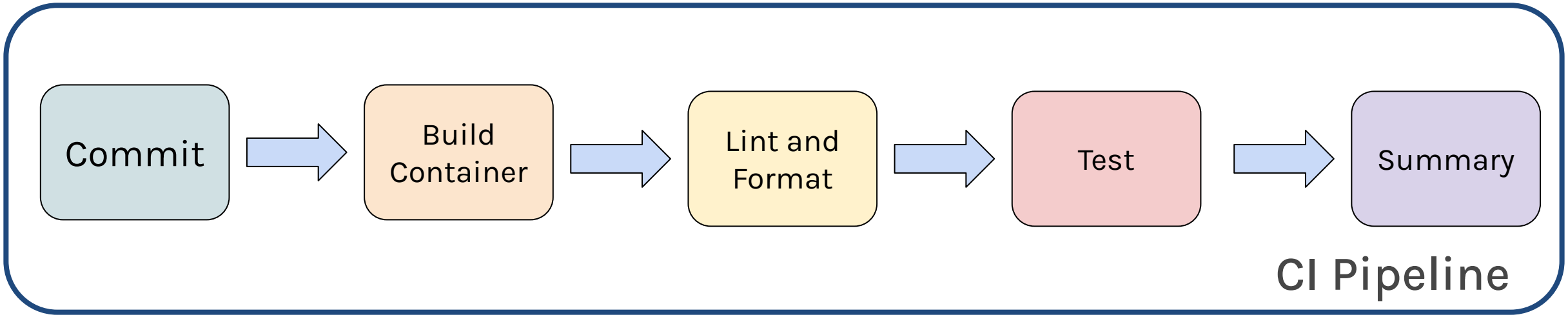
So far, we've talked about what Continuous Integration means.

But now, let's make it real!

We'll see how to use GitHub Actions to build a CI pipeline that runs tests, linters, and validations automatically every time we push or open a pull request.

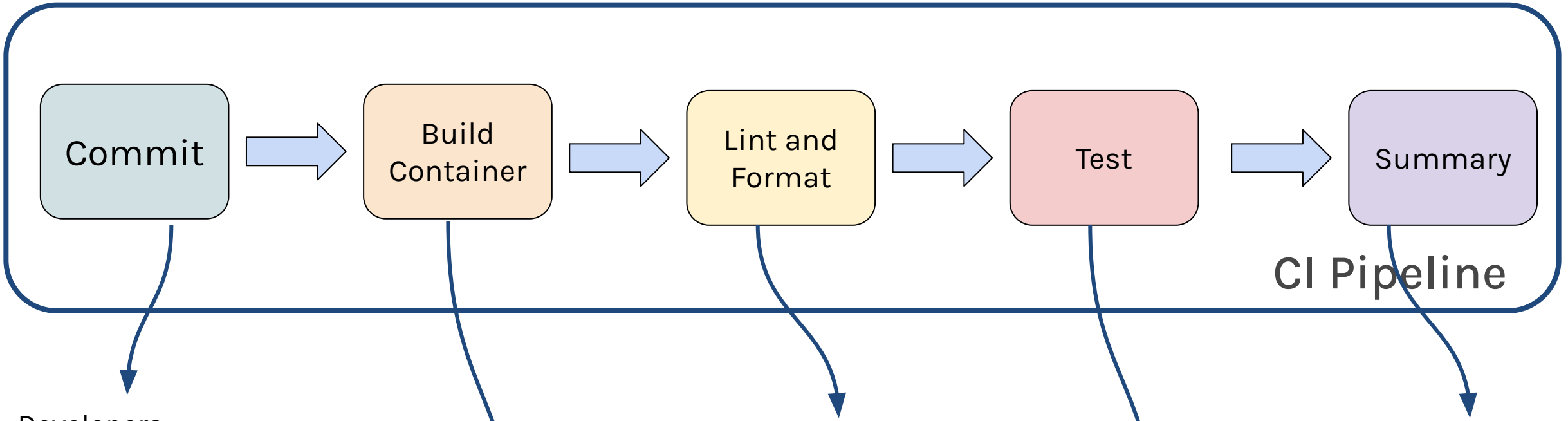
Continuous Integration (CI): Example

The following is the CI pipeline that we have created:



Let's break down the pipeline and understand what we are doing.

Continuous Integration (CI): Example



Developers push code changes to the repo, triggering the CI pipeline.

Builds a Docker image to ensure environment consistency and detect build issues.

Runs tools like Black and Flake8 to check code style and quality.

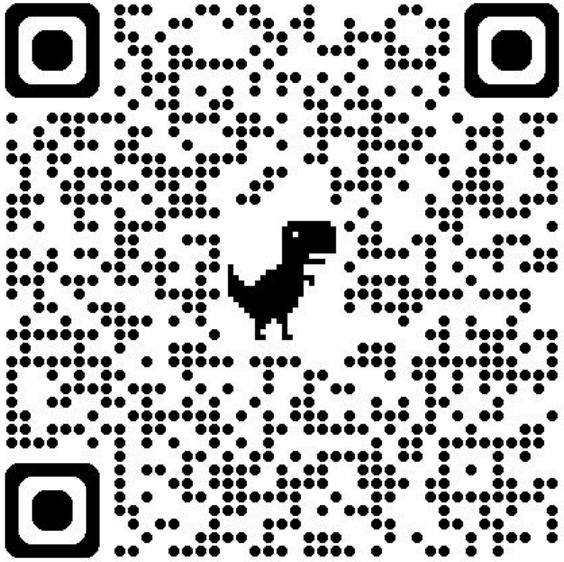
Executes automated unit, integration, and system tests.

Collects results and reports pass/fail status back to developers.

Continuous Integration (CI): Example

Now that we understand what the workflow does,

Let's jump into the actual GitHub Actions configuration and see how we can run it ourselves.

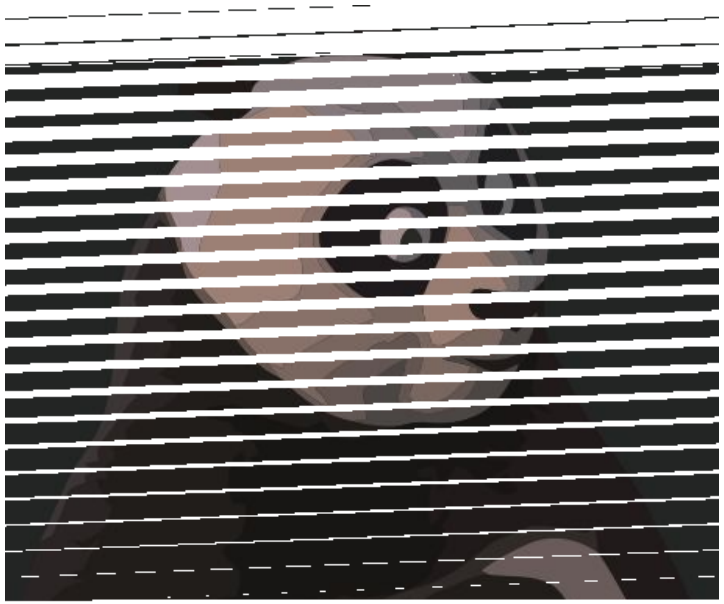


https://github.com/dlops-io/cheese-app-ci-cd/blob/main/README.md#part-5-github-actions-remote-cicd

The Local Connundrum

You might wonder, we already run linters and tests locally using pre-commit hooks.

So why are we doing it again in the CI pipeline?



The Local Connundrum

```
git commit --no-verify -m "quick fix"
```

This is why!



Still confused?

The Local Connundrum

```
git commit --no-verify -m "quick fix"
```

The --no-verify flag skips all local pre-commit checks.

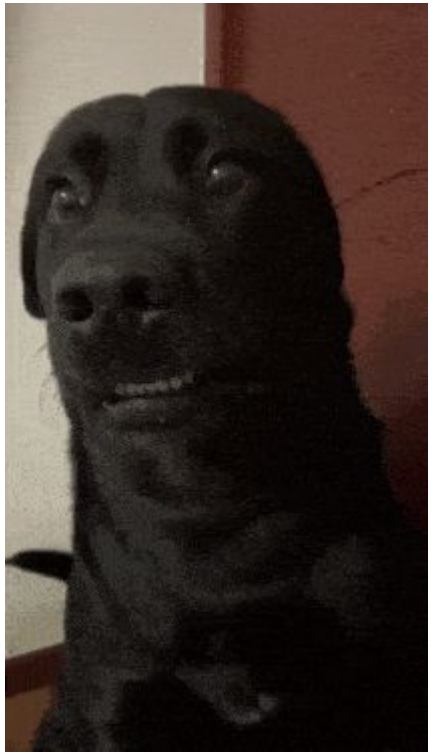
Sometimes, developers use this flag to skip local tests or linters, intentionally or by accident.

When that happens, issues can slip through, and the only way to catch them is by running checks again on the remote CI server.

Outline

- Remote Git Hooks
- Github Webhooks
- Continuous Integration
- **Continuous Deployment** (Lec21)

Continuous Deployment



Wait… We

haven’t even

learned

Deployment

Yet!

Don’t worry — we’ll get there soon!

Continuous Deployment

For now, just remember what CD aims to do:
automate the release process once everything has been tested and approved.





Thank you